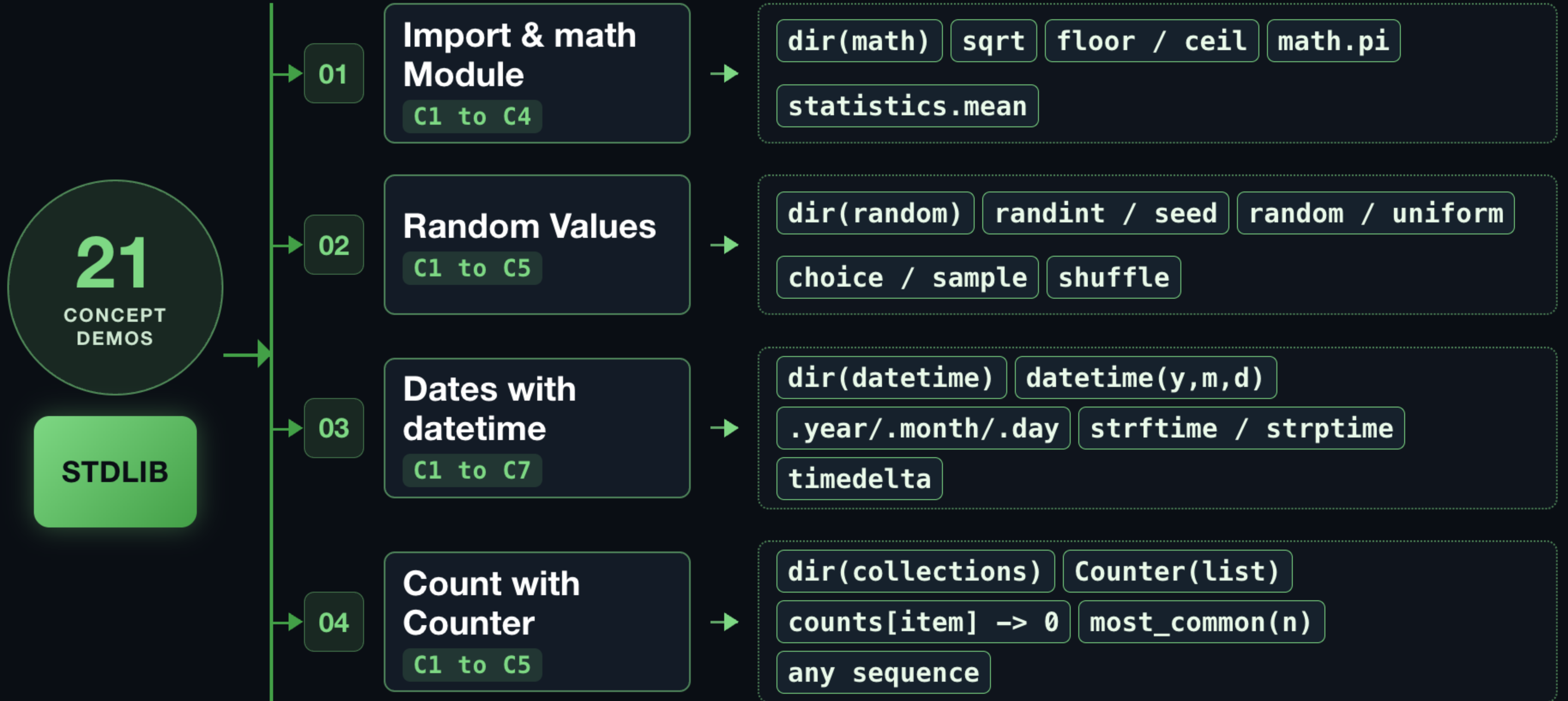


Section 11: Standard Library Essentials **Concept Map**

The full path through this section - we open here before each concept, then dive in.



Std Library: Import a Module and Use **math**

The **standard library** is a set of ready-made modules that come with Python; **import** brings a module like **math** into your program so you can use its tools.

#	CONCEPT	SAMPLE CODE
1	Square root <code>math.sqrt()</code> Get the square root of a number.	<code>math.sqrt(144)</code>
2	Round down and up <code>floor()</code> <code>ceil()</code> Round to the nearest whole number, down or up.	<code>math.floor(4.7), math.ceil(4.2)</code>
3	The pi constant <code>math.pi</code> Use pi without typing the digits yourself.	<code>math.pi</code>
4	Average of a list <code>statistics.mean()</code> Get the mean of a list of numbers.	<code>statistics.mean(scores)</code>

Std Library: Random Values with **random**

The **random** module produces random values - whole numbers, decimals, a random pick from a list, or a shuffled order.

#	CONCEPT	SAMPLE CODE
1	Repeatable runs <code>seed()</code> Fix the sequence so every run matches.	<code>random.seed(42)</code>
2	Whole number in a range <code>randint()</code> An integer from a to b, both ends included.	<code>random.randint(1, 6)</code>
3	Random decimal <code>random()</code> A float from 0.0 up to but not 1.0.	<code>random.random()</code>
4	Pick from a list <code>choice()</code> Grab one random item, no index needed.	<code>random.choice(names)</code>
5	Shuffle in place <code>shuffle()</code> Reorder the list itself, not a copy.	<code>random.shuffle(deck)</code>

Std Library: Dates with **datetime**

The **datetime** module stores a specific date and time as an object you can read, format, and do date math on.

#	CONCEPT	SAMPLE CODE
1	Build a date <code>datetime()</code> Store a specific date you can use in code.	<code>event = datetime(2026, 6, 17)</code>
2	Read date parts <code>.year</code> Pull the year, month, or day number out.	<code>print("year:", event.year)</code>
3	Format as text <code>strftime()</code> Show a date as friendly, readable text.	<code>event.strftime("%d %B %Y")</code>
4	Add or subtract time <code>timedelta()</code> Add days to a date to jump forward in time.	<code>event + timedelta(days=7)</code>
5	Parse text to date <code>strptime()</code> Read a date out of a text string.	<code>datetime.strptime("2026-06-17", "%Y-%m-%d")</code>
6	Days between dates Subtract two dates; read <code>.days</code> for the gap.	<code>(event - today).days</code>

Std Library: Count Items with Counter

Counter is a collections type that counts how many times each item appears in a sequence - all in one step.

#	CONCEPT	SAMPLE CODE
1	Count every item <code>Counter()</code> One step tallies each item in a sequence.	<pre>counts = Counter(votes)</pre>
2	Look up a count <code>counts[key]</code> A missing key returns 0, not an error.	<pre>print("votes for Joe:", counts["Joe"])</pre>
3	Top items <code>most_common(n)</code> Returns the n highest counts, biggest first.	<pre>counts.most_common(2)</pre>
4	Count any sequence <code>Counter("...")</code> Even counts each letter in a word.	<pre>letter_counts = Counter("banana")</pre>